A

Project Report

on

CAMPUS RESOURCE SHARING PLATFORM

BTECH-IT, SEM VI

Prepared By:
Kruti Popat (IT-112)
Nisarg Patel (IT-101)

Guided By:
Prof. Anand K. Patel
Dept. of Information Technology

Department of Information Technology
Faculty of technology,
Dharmsinh Desai University
College road, Nadiad- 387001

April, 2026

CANDIDATE'S DECLARATION

We declare that 6th semester report entitled
"Campus Resource Sharing Platform" is our own work conducted under the supervision of the guide Mr. Anand K. Patel.

We further declare that to the best of our knowledge the report for B.Tech. VI semester does not contain part of the work which has been submitted either in this or any other university without proper citation.

Candidate's Signature
Candidate's Name: Kruti Popat
Student ID: 23ITUOS107

Candidate's Signature
Candidate's Name: Nisarg Patel
Student ID: 23ITUOS154

DHARMSINH DESAI UNIVERSITY
NADIAD-387001, GUJARAT

CERTIFICATE

This is to certify that the project carried out in the subject of Project-I, entitled "Campus Resource Sharing Platform" and recorded in this report is a bonafide report of work of

1) Kruti Popat Roll No. IT112 ID No: 23ITUOS107

2) Nisarg Patel Roll No. IT101 ID No: 23ITUOS154

of Department of Information Technology, semester VI . They involved in Project work during academic year 2025 - 2026.

Prof. Anand K. Patel
(Project Guide),
Department of Information Technology,
Faculty of Technology,
Dharmsinh Desai University, Nadiad
Date:

Prof. (Dr.) V. K. Dabhi
Head , Department of Information Technology,
Faculty of Technology,
Dharmsinh Desai University, Nadiad
Date:

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to the Department of Information Technology for providing us the opportunity and necessary resources to complete this project successfully. The support, guidance, and encouragement provided by the department played a significant role in the completion of this work.

We are also thankful to the faculty members of the department for their valuable suggestions, continuous motivation, and academic support throughout the project. Their guidance helped us understand the concepts and improve the quality of our work.

Finally, we would like to thank everyone who directly or indirectly contributed to the successful completion of this project.

1. **Kruti Popat**
2. **Nisarg Patel**

TABLE OF CONTENTS

Title **Page No**

ABSTRACT…….vii

TABLES……..
1. List of Figures…….ix
2. List of Tables…….. x

CHAPTER-1……. 01

CHAPTER-2…….. 03

CHAPTER-3……. 06

CHAPTER-4……. 09

CHAPTER-5……. 14

CHAPTER-6……. 19

CHAPTER-7……. 20

CHAPTER-8……. 23

CHAPTER-9……. 25

CHAPTER-10…… 26

REFERENCES……27

DETAILS OF CHAPTERS

1. INTRODUCTION 01

1.1 Project Details..... 01

1.2 Purpose..... 01

1.3 Scope 01

1.4 Objective 01

1.5 Technology and Literature Review 02

2. PROJECT MANAGEMENT.....03

2.1 Feasibility Study 03

2.1.1 Technical Feasibility 03

2.1.2 Time Schedule Feasibility 03

2.1.3 Operational Feasibility 03

2.1.4 Implementation Feasibility 03

2.2 Project Planning 03

2.2.1 Project Development Approach and Justification.....03

2.2.2 Project Plan 04

2.2.3 Milestones and Deliverables 04

2.2.4 Roles and Responsibilities 04

2.2.5 Group Dependencies 04

2.3 Project Scheduling 04

3. SYSTEM REQUIREMENTS STUDY 06

3.1 Study of Current System 06

3.2 Problems and Weaknesses of Current System 06

3.3 User Characteristics..... 06

3.4 Hardware and Software Requirements 06

3.5 Constraints 07

3.5.1 Regulatory Policies 07

3.5.2 Hardware Limitations 07
3.5.3 Interfaces to Other Applications 07
3.5.4 Parallel Operations 07
3.5.5 Higher Order Language Requirements 07
3.5.6 Reliability Requirements 07
3.5.7 Criticality of the Application 07
3.5.8 Safety and Security Consideration 08
3.6 Assumptions and Dependencies 08
4. SYSTEM ANALYSIS 09
4.1 Requirements of New System (SRS) 09
4.1.1 User Requirements 09
4.1.2 System Requirements 09
4.2 Features of New System 09
4.3 Navigation Chart 09
4.4 Class Diagram 10
4.5 System Activity 11
4.6 Sequence Diagram 12
4.7 Data Modeling..... 12
4.7.1 Data Dictionary……12
4.7.2 ER Diagram……13
5. SYSTEM DESIGN..... 14
5.1 System Architecture Study.....14
5.1.1 Component Diagram 16
5.1.2 Deployment Diagram 16
5.2 Database Design.....17
5.2.1 Table and Relationship 17
5.2.2 Logical Description of Data 17
5.3 Input/Output and Interface Design 17
5.3.1 State Transition/UML Diagram 18

5.3.2 Sample of Forms, Reports and Interface.....18

6. IMPLEMENTATION PLANNING 19

6.1 Implementation Environment 19

6.2 Program/Modules Specification 19

6.3 Coding Standards 19

7. TESTING.....20

7.1 Testing Plan 20

7.2 Testing Strategy 20

7.3 Testing Methods 20

7.4 Test Cases 20

7.4.1 Purpose 20

7.4.2 Required Input 20

7.4.3 Expected Result 21

8. USER MANUAL 23

9. LIMITATION AND FUTURE ENHANCEMENT 30

9.1 Current Limitations 30

9.2 Future Enhancements.....30

10. CONCLUSION AND DISCUSSION 31

10.1 Conclusions and Future enhancements 31

10.2 Discussion 31

10.2.1 Self-Analysis of Project Viabilities 31

10.2.2 Problems Encountered and Possible Solutions 31

10.2.3 Summary of Project Work 31

ABSTRACT

The Campus Resource Sharing Platform is a web-based system designed to facilitate the sharing and exchange of resources within a college campus. In many institutions, students and faculty often possess useful resources such as books, electronics, furniture, or sports equipment that remain unused. However, there is usually no organized system through which these resources can be easily shared or accessed by others in the campus community. This project aims to solve that problem by providing a centralized platform where users can list, discover, and request resources.

The system allows registered users to add resources they wish to sell, exchange, rent, or donate. Other users can browse available resources by category, send requests to the owners, and communicate through a built-in chat feature. The platform also includes email OTP verification for secure user registration, a request management system, transaction verification by both parties, and a user rating system to maintain trust and reliability among users.

The application is developed using a full-stack approach with Spring Boot for backend services, MySQL for database management, and HTML, CSS, and JavaScript for the frontend interface. The system is designed to be user-friendly, secure, and efficient, enabling easy resource discovery and communication among users.

Overall, the Campus Resource Sharing Platform promotes efficient utilization of resources, reduces unnecessary purchases, and encourages collaboration within the campus community by providing a simple and reliable digital solution for peer-to-peer resource sharing.

List of Tables

No.	Name	Page No.
1.1	Technology Stack Summary	02
2.1	Project Schedule and Phases	04
3.1	Minimum Hardware and Software Requirements	06
7.1	Test Case Inputs	20
7.2	Test Case Expected Results	21

List of Figures

No.	Name	Pg. no.
Fig 4.1	Navigation Chart	09
Fig 4.2	Class Diagram (Analysis Level)	10
Fig 4.3	Use Case Diagram	11
Fig 4.4	Sequence Diagram (Analysis Level)	12
Fig 4.5	ER Diagram	13
Fig 5.1	Class Diagram (Design Level)	14
Fig 5.2	Sequence Diagram (Design Level)	15
Fig 5.3	Component Diagram	16
Fig 5.4	Deployment Diagram	16
Fig 5.5	State Transition Diagram	17

CHAPTER 1 INTRODUCTION

1.1 PROJECT DETAILS: BROAD SPECIFICATIONS OF THE WORK ENTRUSTED

The Campus Resource Sharing Platform is a full-stack web application designed to facilitate the sharing of resources within a campus community. The work entails development of a complete system comprising a RESTful backend built with Spring Boot, a MySQL database for persistent storage, and a responsive frontend using HTML, CSS, and JavaScript. The platform enables students and faculty to list resources (books, electronics, furniture, sports equipment) for sale, exchange, rent, or donation, and allows other users to request these resources, communicate via an in-built chat system, and complete transactions with mutual verification.

1.2 PURPOSE

The purpose of this project is to create a centralized platform that promotes sustainable resource utilization within educational institutions. By enabling peer-to-peer sharing, the system reduces unnecessary purchases, supports a circular economy, and strengthens the campus community. The platform aims to make it easy for users to discover, request, and share resources in a trusted, verifiable manner.

1.3 SCOPE

The scope of the project covers user registration with email OTP verification, authentication, resource management (add, update, delete, browse), request lifecycle management (create, approve, reject, cancel), real-time chat between parties, mutual verification for transaction completion, user rating system, and user profile management. The system operates within a single institutional domain (campus) and does not extend to external marketplaces or payment gateways.

1.4 OBJECTIVE (SCOPE – WHAT IT CAN DO AND CANNOT DO)

What the system can do: Register and authenticate users with email verification; list resources under categories (Books, Electronics, Furniture, Sports, Other); support four transaction types (Sell, Exchange, Rent, Donate); filter resources by category; send and manage resource requests; approve or reject requests; enable chat between owner and requester; verify handover and receipt to complete transactions; rate users after completed transactions; display user profile and average rating.

What the system cannot do: Process online payments (no payment gateway integration); support external users outside the campus; provide real-time push notifications (uses polling for chat); handle image uploads for resources (text-only descriptions); enforce physical handover (relies on user verification).

1.5 TECHNOLOGY AND LITERATURE REVIEW

Spring Boot is an opinionated framework for building production-ready Java applications. It provides auto-configuration, embedded Tomcat, and seamless integration with Spring Data JPA. The framework reduces boilerplate and accelerates development (Walls, 2016).

MySQL is a widely-used relational database management system offering ACID compliance, scalability, and strong community support. It is suitable for structured data with relationships (Schwartz et al., 2012).

REST (Representational State Transfer) is an architectural style for designing networked applications. RESTful APIs use HTTP methods (GET, POST, PUT, DELETE) and JSON for data exchange (Fielding, 2000).

JPA/Hibernate provides Object-Relational Mapping (ORM), enabling developers to work with Java objects instead of raw SQL. Hibernate is the reference implementation of JPA (Bauer & King, 2015).

Table 1.1 Technology Stack Summary

Layer	**Technology**	**Version**	**Purpose**
Backend	Spring Boot	3.2.0	REST API, dependency injection
Database	MySQL	8.0	Persistent storage
ORM	Hibernate / JPA	6.x	Object-relational mapping
Frontend	HTML5, CSS3, JS	ES6+	User interface
Build	Maven	3.6+	Dependency management

CHAPTER 2 PROJECT MANAGEMENT

2.1 FEASIBILITY STUDY

2.1.1 Technical Feasibility

The chosen technology stack (Java 17, Spring Boot 3.2, MySQL 8, HTML5 / CSS3 / JavaScript) is mature, well-documented, and widely adopted. All components are open-source and freely available. The development team has access to necessary hardware (laptop, internet) and software (IDE, database server). Technical feasibility is high.

2.1.2 Time Schedule Feasibility

The project was planned for completion within one academic semester. Core features (registration, login, resources, requests) were prioritized; additional features (OTP verification, ratings, chat) were added iteratively. The incremental approach ensured time feasibility.

2.1.3 Operational Feasibility

The system is designed for non-technical users (students, faculty). The interface is intuitive with clear labels and minimal learning curve. The application runs on standard web browsers and does not require special training. Operational feasibility is high.

2.1.4 Implementation Feasibility

Implementation is feasible using standard development practices. The layered architecture (Controller-Service-Repository) allows parallel development. Maven manages dependencies; H2 can be used for development with MySQL for production. Implementation feasibility is confirmed.

2.2 PROJECT PLANNING

2.2.1 Project Development Approach and Justification

An incremental/iterative approach was adopted. Each iteration delivered a working increment: first authentication and resources, then requests and chat, then OTP verification and ratings. This approach allows early feedback, reduces risk, and enables gradual integration. Agile practices (short cycles, continuous integration) were followed where applicable.

2.2.2 Project Plan

- Phase 1: Requirements and database design
- Phase 2: Backend (entities, APIs)
- Phase 3: Frontend (login, dashboard)
- Phase 4: Request and chat features
- Phase 5: OTP, ratings, testing

• Phase 6: Documentation and deployment

2.2.3 Milestones and Deliverables

• M1: Database schema and basic CRUD APIs
• M2: User authentication and resource UI
• M3: Request workflow and chat
• M4: OTP verification and ratings
• M5: Documentation and tested system

2.2.4 Roles and Responsibilities

• Project lead: Coordination, architecture
• Backend developer: APIs, database
• Frontend developer: UI/UX
• Tester: Test cases, bug reporting
• Documenter: Report, user manual

2.2.5 Group Dependencies

Backend APIs must be ready before frontend integration. Database schema must be finalized before entity and repository development. OTP service depends on email configuration.

2.3 PROJECT SCHEDULING

Project Scheduling Chart

Table 2.1 Project Schedule and Phases

Phase	Activities	Duration (Weeks)
1	Requirements gathering, database design	1–2
2	Backend development (entities, APIs)	3–5
3	Frontend development (UI, forms)	4–6
4	Integration, OTP, ratings	5–6

5	Testing, bug fixes	6–7
6	Documentation, final submission	7–8

CHAPTER 3 SYSTEM REQUIREMENTS STUDY

3.1 STUDY OF CURRENT SYSTEM

Prior to this platform, campus resource sharing relied on informal methods: notice boards, social media groups, word-of-mouth, and manual coordination. There was no centralized system to list, search, or manage resource availability. Users had to post in multiple groups and coordinate via separate messaging apps. No record of transactions or user reputation existed.

3.2 PROBLEMS AND WEAKNESSES OF CURRENT SYSTEM

• Poor discoverability
• no trust or verification mechanism
• fragmented communication
• no transaction history or ratings
• time-consuming and inefficient

3.3 USER CHARACTERISTICS

Primary users are college students and faculty. They are generally tech-savvy, use smartphones and laptops, and are comfortable with web applications. They expect simple navigation, fast responses, and mobile-friendly interfaces. Users may have varying levels of technical expertise; the system must remain accessible to all.

3.4 HARDWARE AND SOFTWARE REQUIREMENTS

Table 3.1 Minimum Hardware and Software Requirements

Component	**Requirement**
Server CPU	2 GHz or higher
Server RAM	4 GB minimum
Storage	500 MB free space
Operating System	Windows / Linux / Mac

Client Browser	Chrome, Firefox, Edge (latest)
Screen Resolution	1366x768 or higher
Java	Version 17 or higher
MySQL	Version 8.0 or higher
Maven	Version 3.6 or higher

3.5 CONSTRAINTS

3.5.1 Regulatory Policies: Data must be handled per institutional policies; personal data (email, phone) stored with consent.

3.5.2 Hardware Limitations: System must run on typical campus infrastructure; no specialized hardware assumed.

3.5.3 Interfaces to Other Applications: Email service (SMTP) for OTP; no payment or external marketplace integration.

3.5.4 Parallel Operations: Multiple users can access the system concurrently; database handles concurrent access.

3.5.5 Higher Order Language Requirements: Java 17; JavaScript ES6+.

3.5.6 Reliability Requirements: System should be available during campus hours; graceful handling of failures.

3.5.7 Criticality of Application: Non-critical; downtime causes inconvenience but no safety impact.

3.5.8 Safety and Security Consideration: Passwords stored (ideally hashed); OTP for email verification; CORS configured for allowed origins.

3.6 ASSUMPTIONS AND DEPENDENCIES

MySQL server is installed and accessible. SMTP credentials for email are configured. Users provide valid email addresses. Campus has stable internet. Users act in good faith; physical handover is not enforced by the system. Browser supports Fetch API and localStorage.

CHAPTER 4 SYSTEM ANALYSIS

4.1 REQUIREMENTS OF NEW SYSTEM (SRS)

4.1.1 User Requirements

• Register with username, email, password, full name, optional phone
• Verify email via OTP before activation
• Login, add resources, browse and filter by category
• Send requests; approve or reject (owners)
• Chat for approved requests; verify handover and receipt
• Rate users after completion; update profile

4.1.2 System Requirements

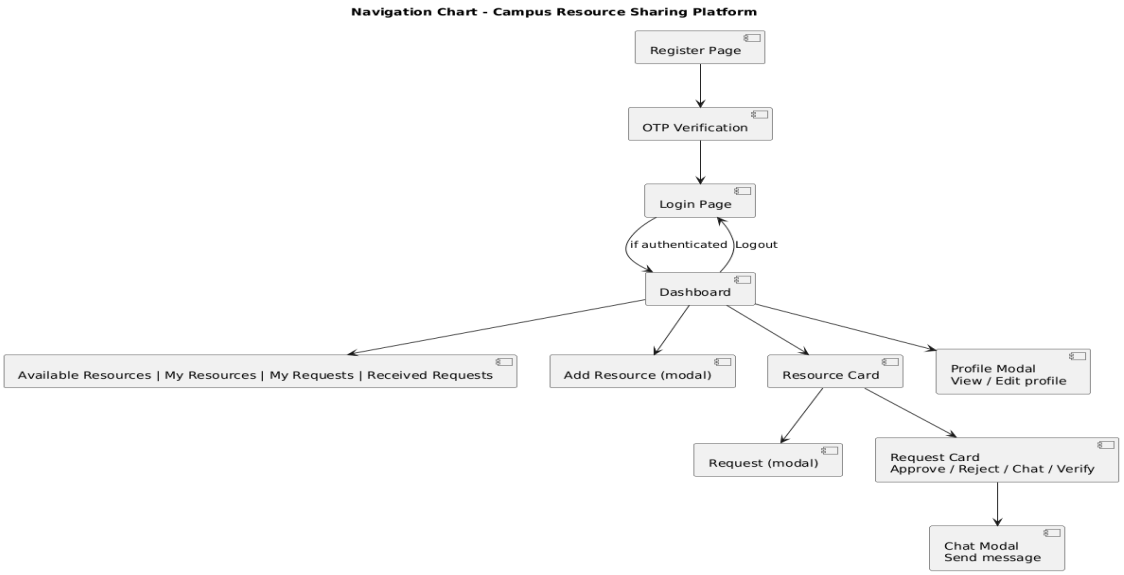
• Validate inputs; store data in MySQL; expose REST APIs
• Support concurrent sessions; send OTP via SMTP
• Enforce referential integrity; respond within acceptable time

4.2 FEATURES OF NEW SYSTEM

User registration with OTP verification; Login/Logout; Add, update, delete resources; Browse and filter resources; Create, approve, reject, cancel requests; Real-time chat (polling-based); Mutual verification for transaction completion; User rating (1–5 stars with optional comment); User profile with average rating; Responsive web interface.

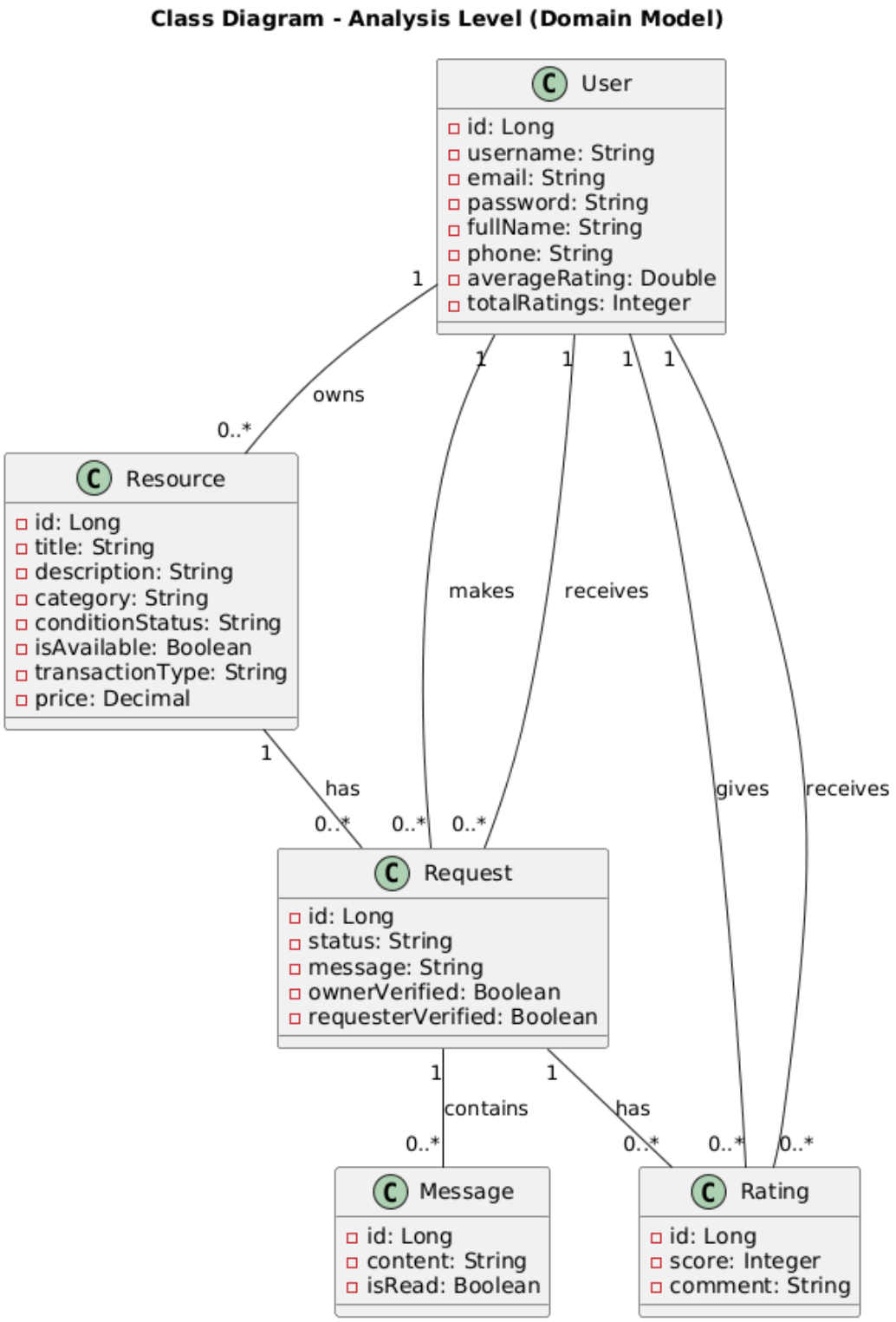
4.3

FIG 4.1 NAVIGATION CHART



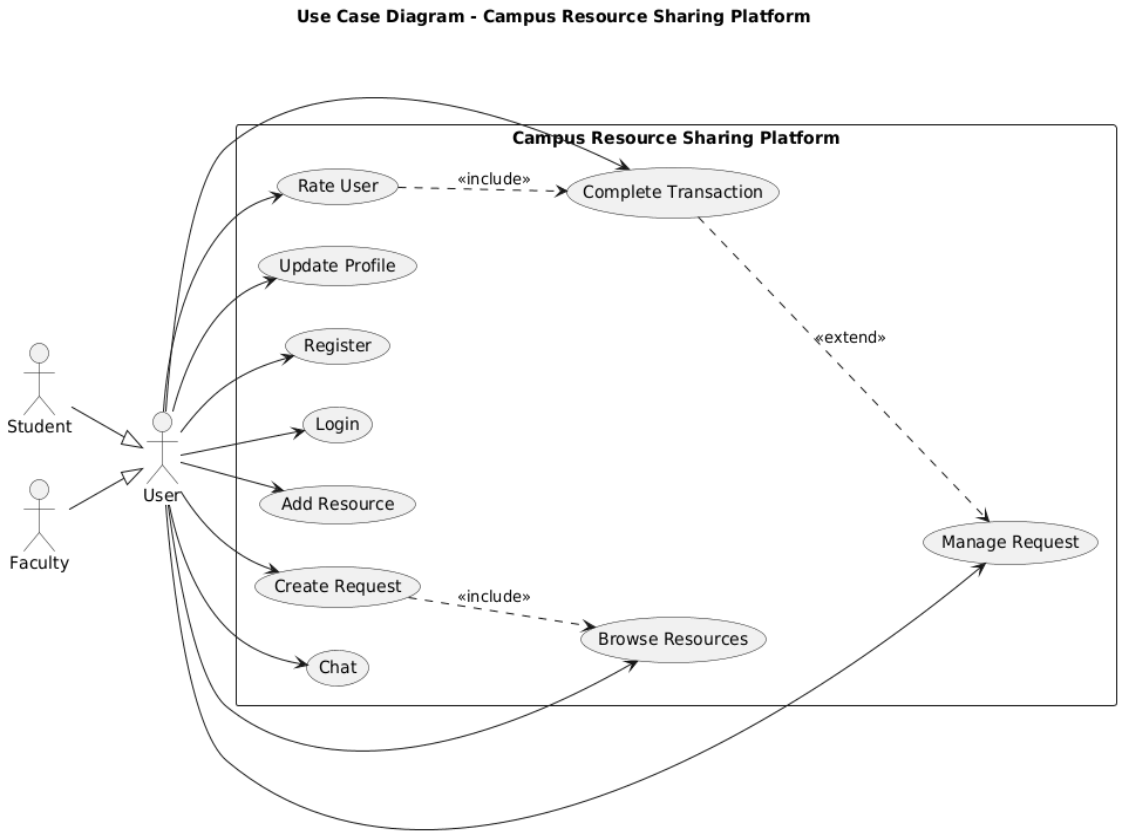
4.4

FIG 4.2 CLASS DIAGRAM (ANALYSIS LEVEL)



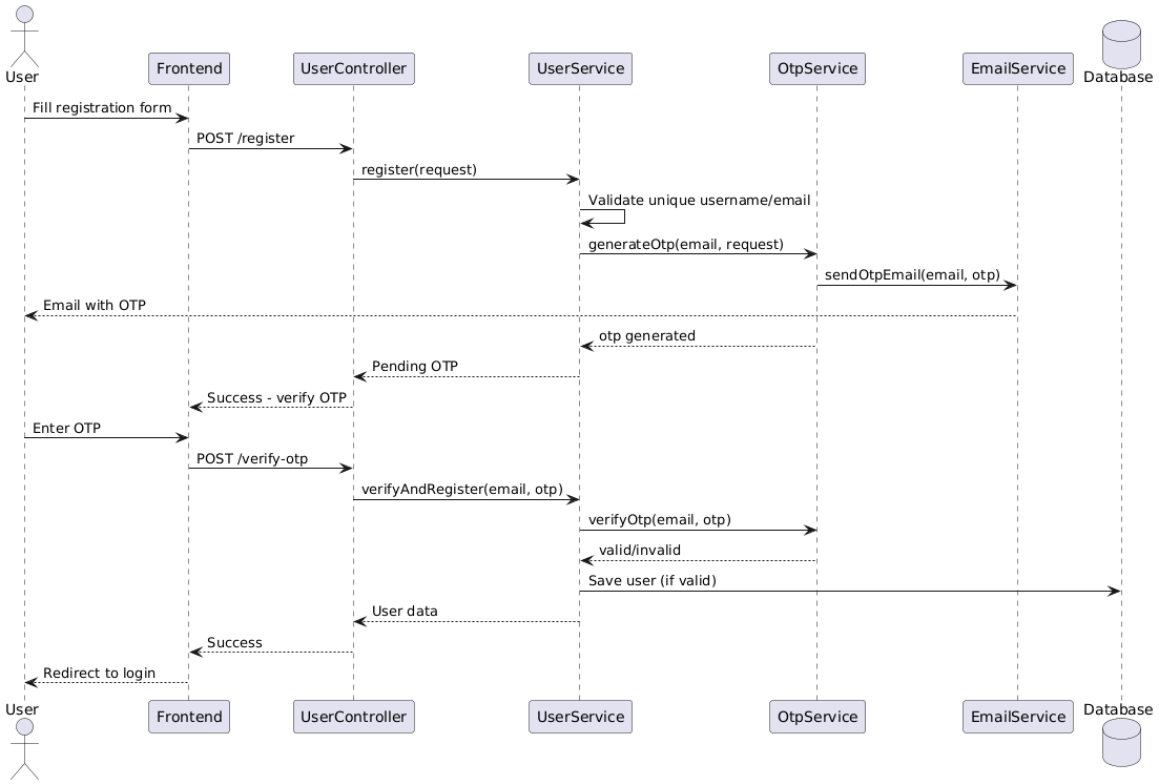
4.5

FIG 4.3 USE CASE DIAGRAM



4.6

FIG 4.4 SEQUENCE DIAGRAM (ANALYSIS LEVEL)



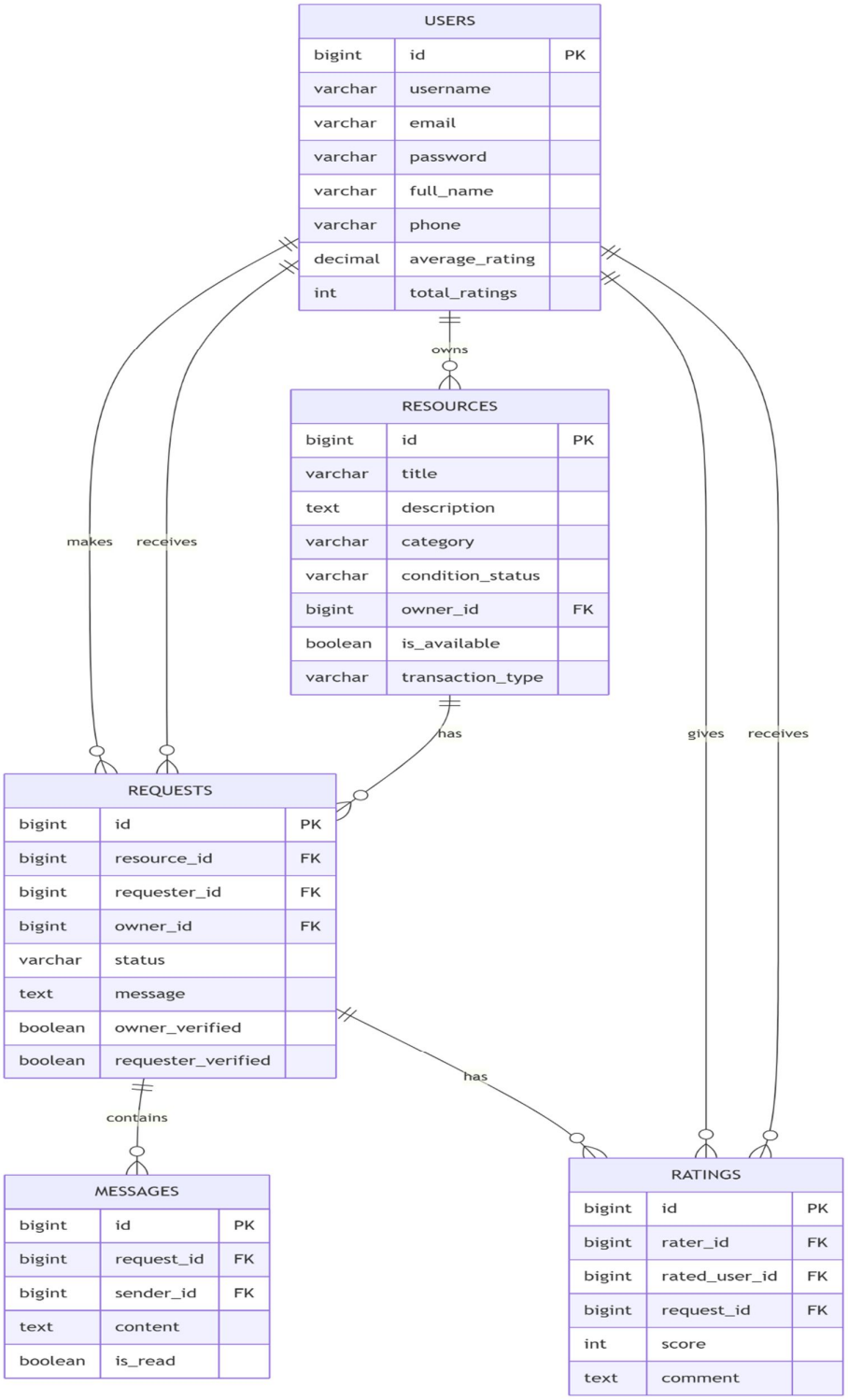
4.7 DATA MODELING

4.7.1 Data Dictionary

- **users**: id, username, email, password, full_name, phone, bio, department, year, average_rating, total_ratings, created_at, updated_at
- **resources**: id, title, description, category, condition_status, owner_id, is_available, transaction_type, price, rental_duration, exchange_description, created_at, updated_at
- **requests**: id, resource_id, requester_id, owner_id, status, message, owner_verified, requester_verified, created_at, updated_at
- **messages**: id, request_id, sender_id, content, is_read, created_at
- **ratings**: id, rater_id, rated_user_id, request_id, score, comment, created_at

4.7.2

FIG 4.4 ER DIAGRAM



CHAPTER 5 SYSTEM DESIGN

5.1 SYSTEM ARCHITECTURE DESIGN (OO/MVC APPROACH)

FIG 5.1 CLASS DIAGRAM (DESIGN LEVEL, MVC)

FIG 5.2 SEQUENCE DIAGRAMS (DESIGN LEVEL, MVC)

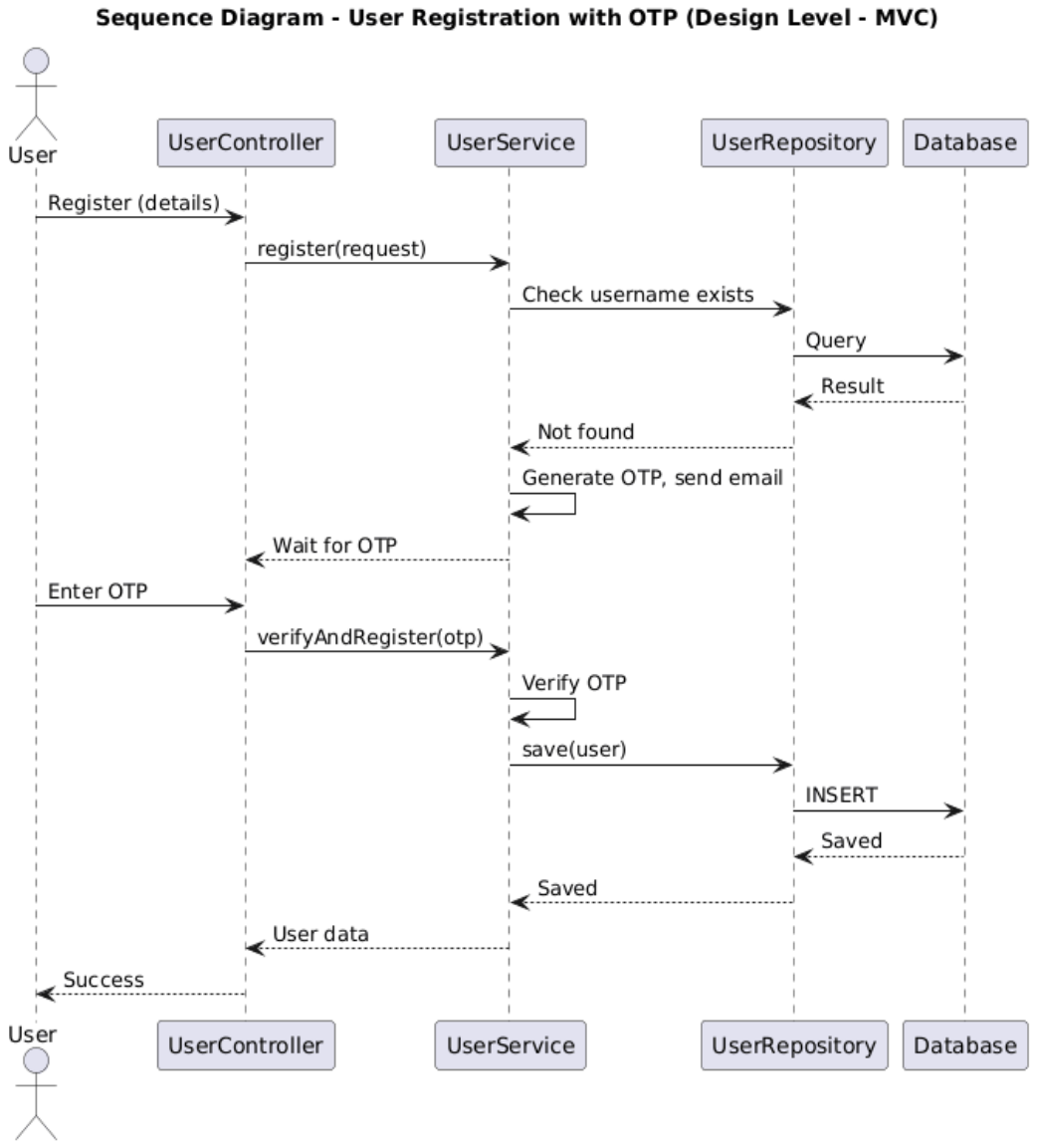


FIG 5.3 COMPONENT DIAGRAM

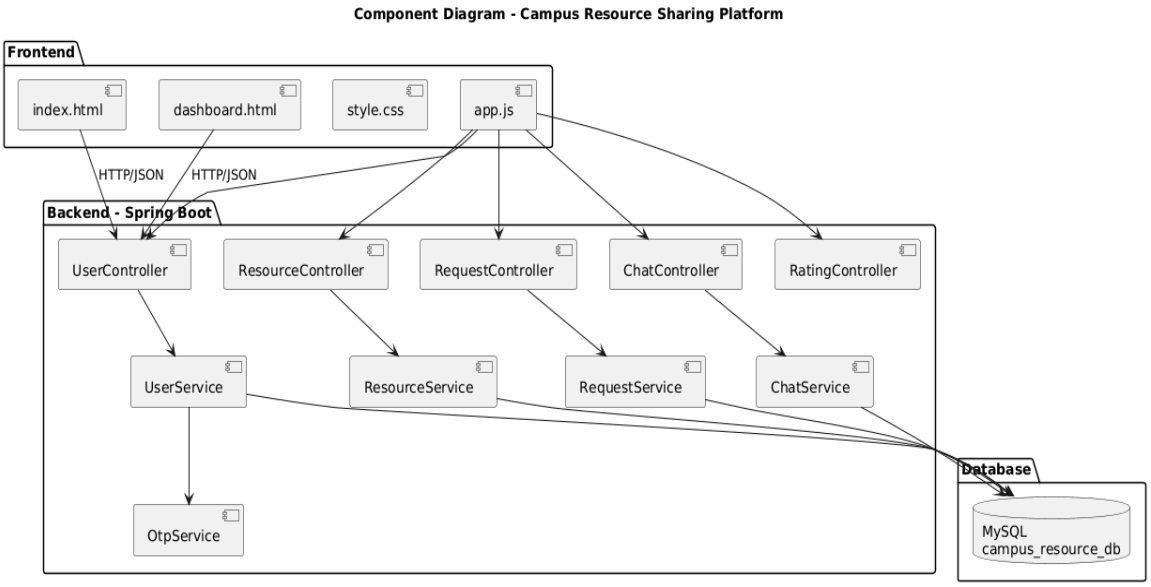
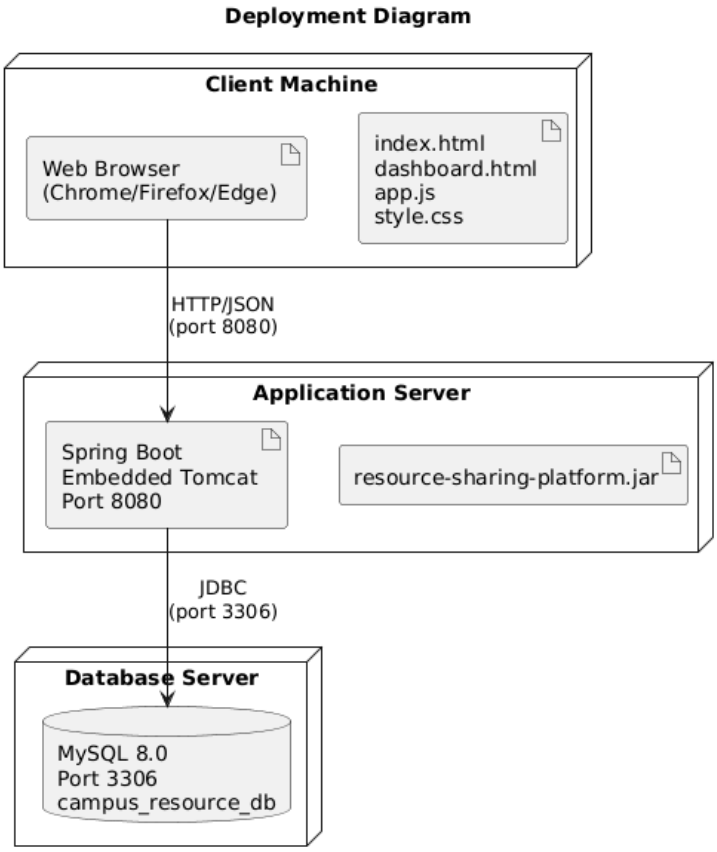


FIG 5.4 DEPLOYMENT DIAGRAM



5.2 DATABASE DESIGN / DATA STRUCTURE DESIGN

5.2.1 Table and Relationship

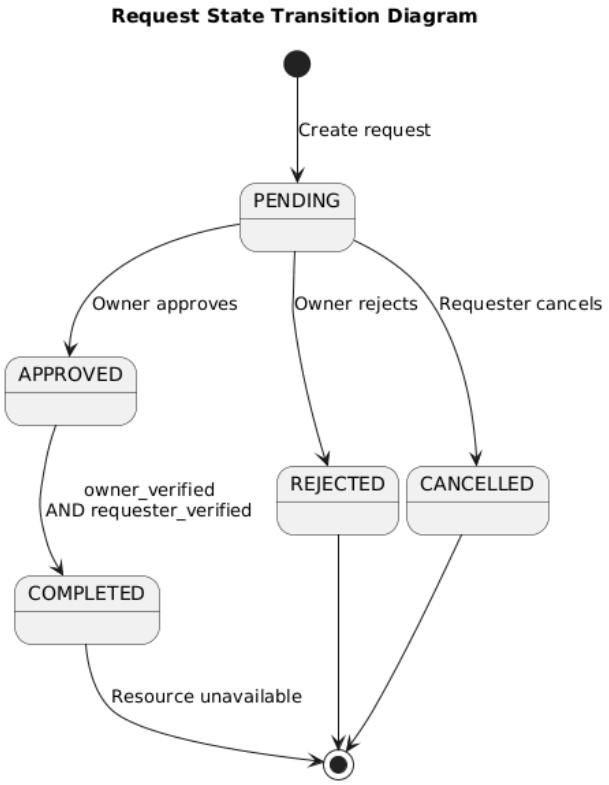
users (PK: id); resources (PK: id, FK: owner_id → users); requests (PK: id, FK: resource_id → resources, requester_id → users, owner_id → users); messages (PK: id, FK: request_id → requests, sender_id → users); ratings (PK: id, FK: rater_id → users, rated_user_id → users, request_id → requests). ON DELETE CASCADE on owner_id, resource_id, requester_id, owner_id, request_id, sender_id.

5.2.2 Logical Description of Data

User represents a registered campus member. Resource represents an item listed for sharing. Request represents a request for a resource with status lifecycle (PENDING, APPROVED, REJECTED, CANCELLED). Message represents a chat message within a request. Rating represents feedback from one user to another for a completed request.

5.3 INPUT/OUTPUT AND INTERFACE DESIGN

FIG 5.5 STATE TRANSITION DIAGRAM



5.3.2 Samples of Forms, Reports and Interface

Forms: Login (username, password); Register (username, email, password, full name, phone); OTP verification (6-digit OTP); Add Resource (title, description, category, condition, transaction type, price/rental/exchange as applicable); Request (optional message); Rating (score 1–5, optional comment).

Reports: List of available resources; My resources; My requests; Received requests; User profile with average rating.

Interface: Tab-based auth (Login/Register/OTP); Dashboard with filter and grid; Modals for Add Resource, Request, Chat, Profile.

CHAPTER 6 IMPLEMENTATION PLANNING

6.1 IMPLEMENTATION ENVIRONMENT

The system is implemented as a multiuser web application. Multiple users can access the system concurrently from different clients. The frontend provides a graphical user interface (GUI) using HTML5, CSS3, and JavaScript. The backend runs as a standalone Spring Boot application with an embedded Tomcat server. Development environment: Windows/Linux/Mac; IDE: IntelliJ IDEA or Eclipse; Build: Maven.

6.2 PROGRAM/MODULES SPECIFICATION

- **UserController**: register, verifyOtp, login, getUserById, updateProfile
- **ResourceController**: addResource, getAllResources, getResourcesByOwner, updateResource, deleteResource
- **RequestController**: createRequest, approveRequest, rejectRequest, cancelRequest, verifyHandover, verifyReceipt, getMyRequests, getReceivedRequests
- **ChatController**: sendMessage, getMessages, getUnreadCount
- **RatingController**: submitRating, getRatingsForUser
- **Frontend**: handleLogin, handleRegister, handleVerifyOtp, loadResources, filterResources, loadMyResources, loadMyRequests, loadReceivedRequests, handleAddResource, approveRequest, rejectRequest, openChatModal, loadChatMessages, sendChatMessages, verify Handover, verifyReceipt, showProfileModal

6.3 CODING STANDARDS

Java: Follow Java naming conventions (camelCase for methods/variables, PascalCase for classes); use meaningful names; keep methods focused; add comments for complex logic. REST: Use proper HTTP methods (GET, POST, PUT, DELETE); return consistent JSON structure (success, message, data); use appropriate status codes. Frontend: Use semantic HTML; separate structure (HTML), style (CSS), behavior (JS); avoid inline styles and scripts where possible; use async/await for API calls. Database: Use snake_case for column names; define indexes on foreign keys and frequently queried columns.

CHAPTER 7 TESTING

7.1 TESTING PLAN

Unit testing for service layer logic (e.g., OTP verification, rating calculation). Integration testing for API endpoints using MockMvc or RestAssured. Frontend testing: manual verification of user flows. Database testing: constraint validation, cascade behavior. End-to-end testing: complete user journeys (register → verify → login → add resource → request → approve → chat → verify → rate).

7.2 TESTING STRATEGY

Bottom-up approach: test repositories and services first, then controllers, then full stack. Black-box testing for API behavior. Exploratory testing for UI. Regression testing after each feature addition.

7.3 TESTING METHODS

Unit tests: JUnit 5, Mockito. Integration tests: Spring Boot Test, TestContainers or H2 for database. Manual testing: checklist-based verification of forms and workflows.

7.4 TEST CASES

7.4.1 Purpose

Validate registration, login, resource CRUD, request lifecycle, chat, verification, and rating functionality. Ensure error handling and edge cases.

7.4.2 Required Input

Table 7.1 Test Case Inputs

TCID	Test Scenario	Required Input
TC1	User Registration	Valid username, email, password, full name
TC2	User Login	Correct username and password
TC3	Add Resource	Title, description, category, condition, transaction type

TC4	Create Request	Resource ID, requester ID, optional message
TC5	Approve Request	Request ID, owner ID
TC6	Verify Transaction	Owner and requester verification actions
TC7	Submit Rating	Score 1-5, optional comment

7.4.3 Expected Result

Table 7.2 Test Case Expected Results

TC ID	**Expected Result**
TC1	OTP sent; user created after verification
TC2	User data returned; redirect to dashboard
TC3	Resource saved and listed in My Resources
TC4	Request created with PENDING status
TC5	Status APPROVED; chat enabled
TC6	Both verified; resource becomes unavailable

TC7	Rating saved; user average rating updated

CHAPTER 8 USER MANUAL

8.1 GETTING STARTED

Open the application URL in a browser. Ensure backend is running (default port 8080) and frontend is served (e.g., Live Server on port 5500).

8.2 REGISTRATION

Click Register tab. Fill username, email, password (min 6 chars), full name, phone (optional). Click Register. Check email for 6-digit OTP. Enter OTP and click Verify & Complete Registration. OTP expires in 5 minutes; use Resend OTP if needed.

8.3 LOGIN

Enter username and password. Click Login. On success, you are redirected to the dashboard.

8.4 DASHBOARD

View available resources in a grid. Use Filter by Category to narrow results. Click + Add Resource to list a new item. Fill form (title, description, category, condition, transaction type). For Sell: enter price. For Rent: enter price and duration. For Exchange: enter exchange description. Click Add Resource.

8.5 REQUESTING A RESOURCE

Click Request on a resource card. Optionally enter a message. Click Send Request. Track status in My Requests tab.

8.6 MANAGING REQUESTS (OWNER)

Go to Received Requests tab. For each pending request, click Approve or Reject. After approval, Chat and Verify Handover buttons appear.

8.7 CHAT

Click chat on an approved request. Type message and click send or press enter. Messages auto-refresh every 2 seconds.

8.8 COMPLETING A TRANSACTION

After physical handover, owner clicks Verify Handover. Requester clicks Verify Receipt. When both are verified, the transaction is complete and the resource become unavailable.

8.9 RATING

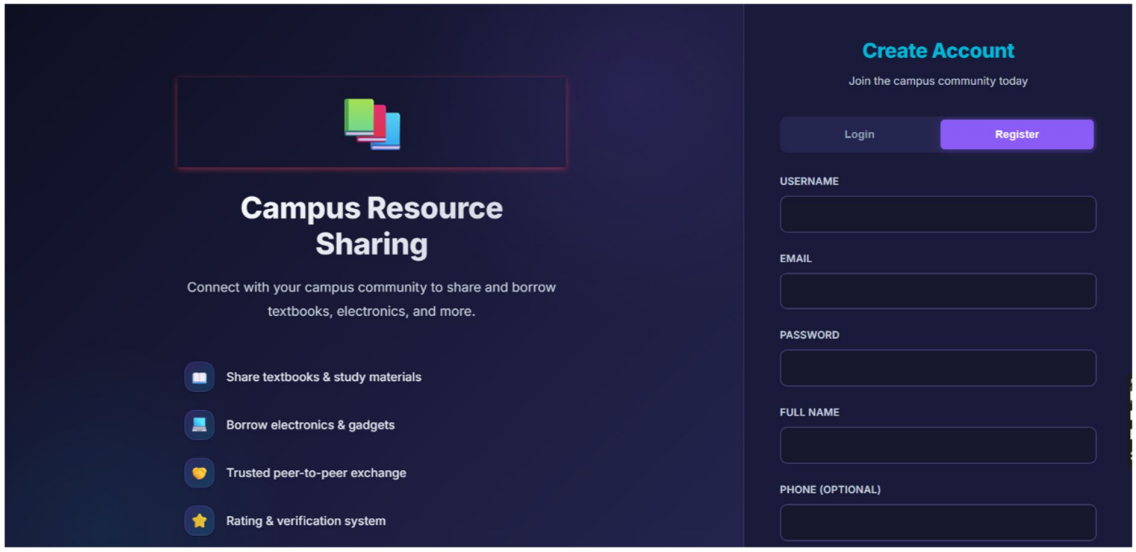
After completion, both parties can rate each other (1–5 stars, optional comment). Rating is shown on the user profile.

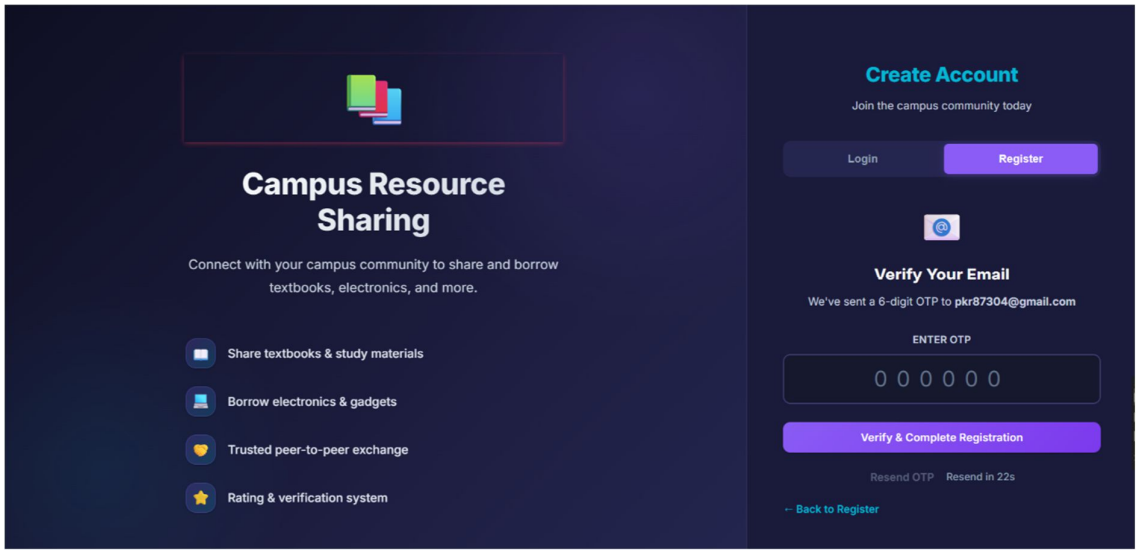
8.10 PROFILE

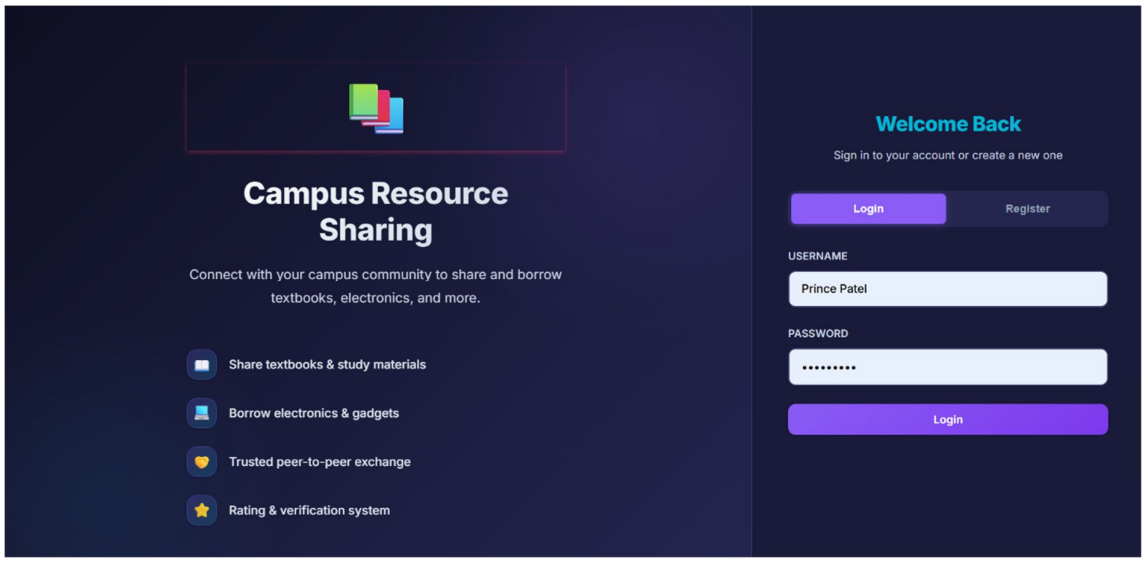
Click Profile to view and edit bio, department, year. Average rating and total ratings are displayed.

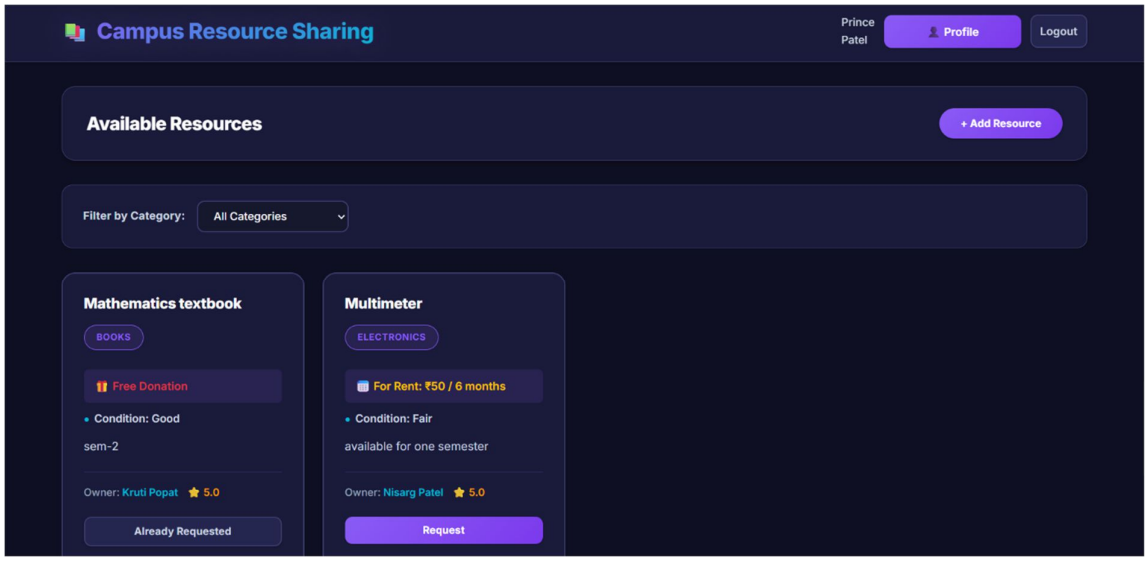
8.11 LOGOUT

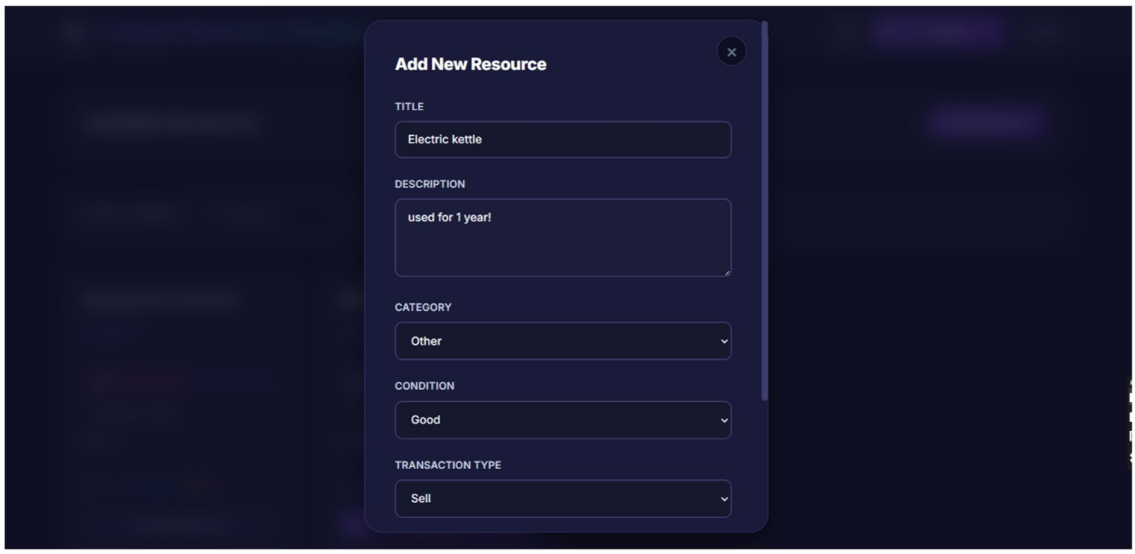
Click Logout to end the session.

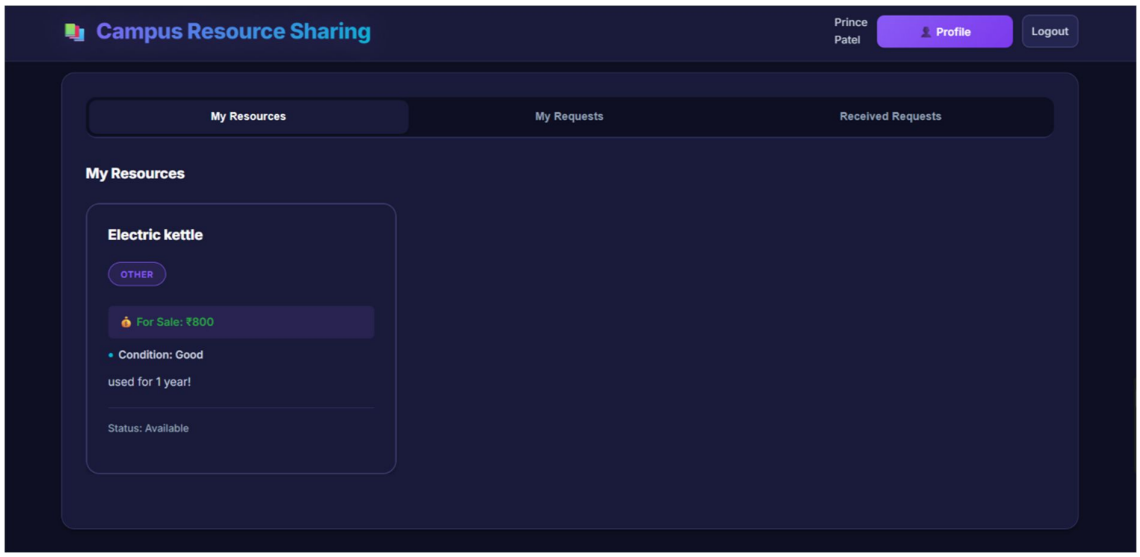


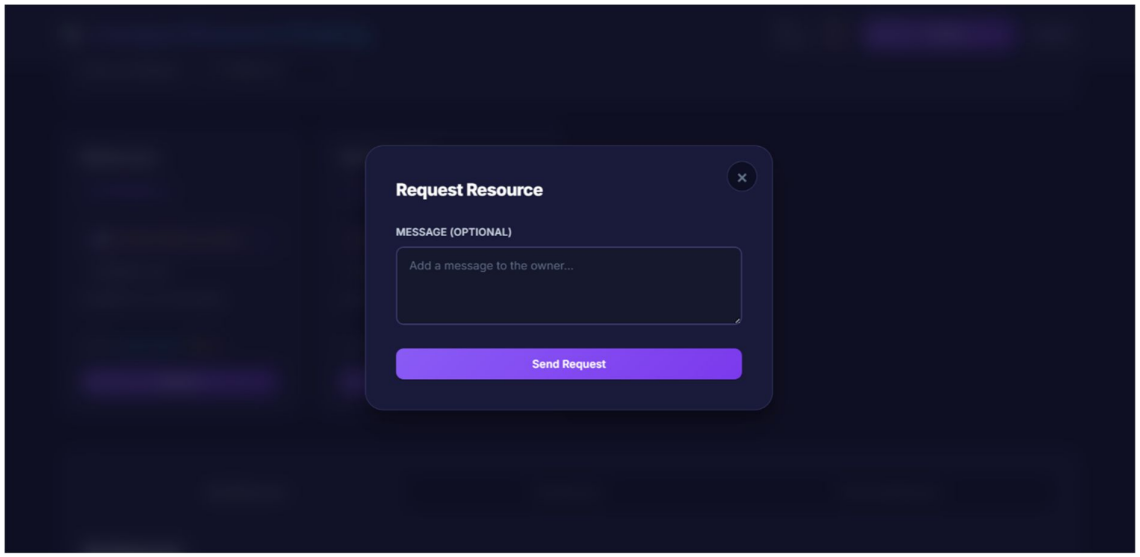


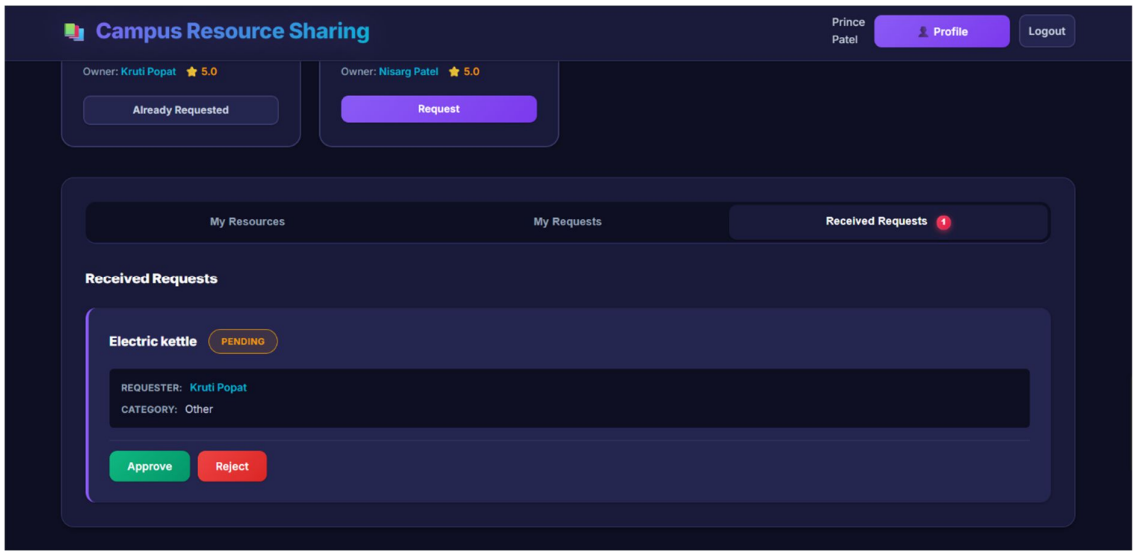

Campus Resource Sharing
Connect with your campus community to share and borrow textbooks, electronics, and more.
Share textbooks & study materials
Borrow electronics & gadgets
Trusted peer-to-peer exchange
Rating & verification system
Welcome Back
Sign in to your account or create a new one
Login
Register
USERNAME
Prince Patel
PASSWORD
Login

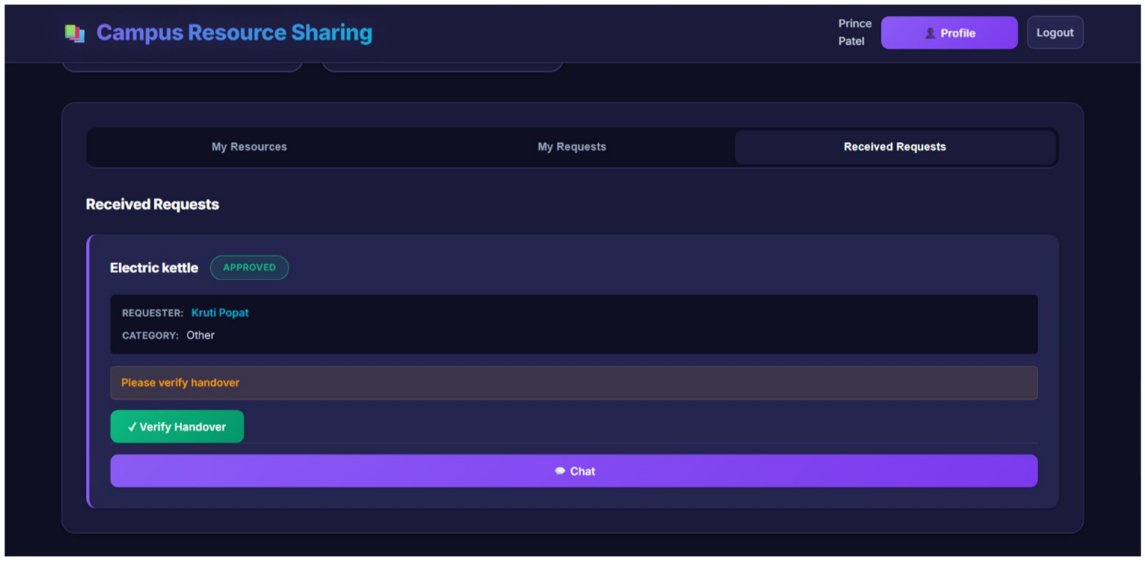

Campus Resource Sharing
Prince Patel
Profile
Logout
Available Resources
+ Add Resource
Filter by Category:
All Categories
Mathematics textbook
BOOKS
Free Donation
Condition: Good
sem-2
Owner: Kruti Popat 5.0
Already Requested
Multimeter
ELECTRONICS
For Rent: ₹50 / 6 months
Condition: Fair
available for one semester
Owner: Nisarg Patel 5.0
Request

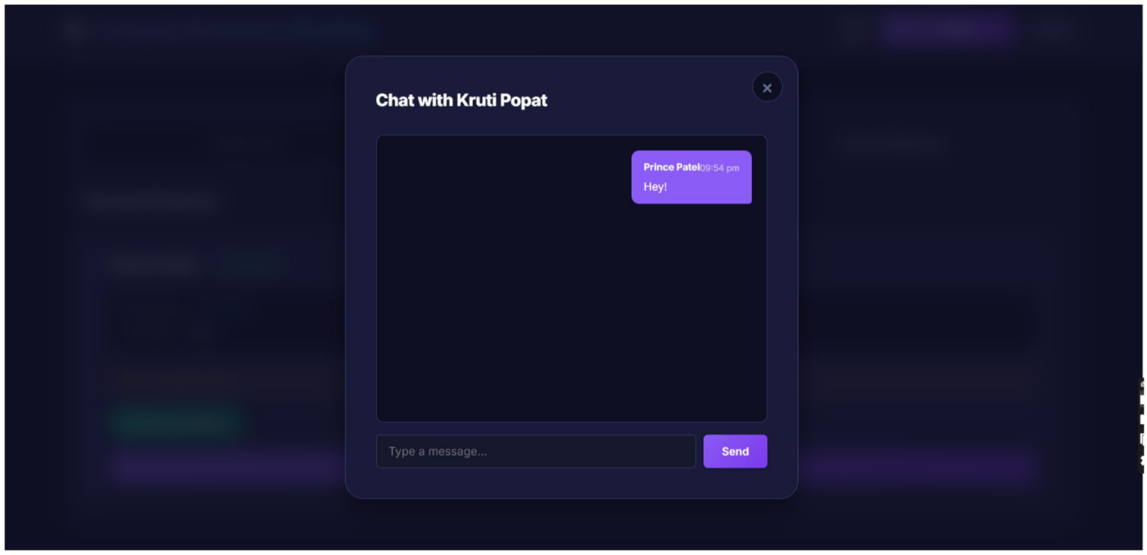

Add New Resource
TITLE
Electric kettle
DESCRIPTION
used for 1 year!
CATEGORY
Other
CONDITION
Good
TRANSACTION TYPE
Sell

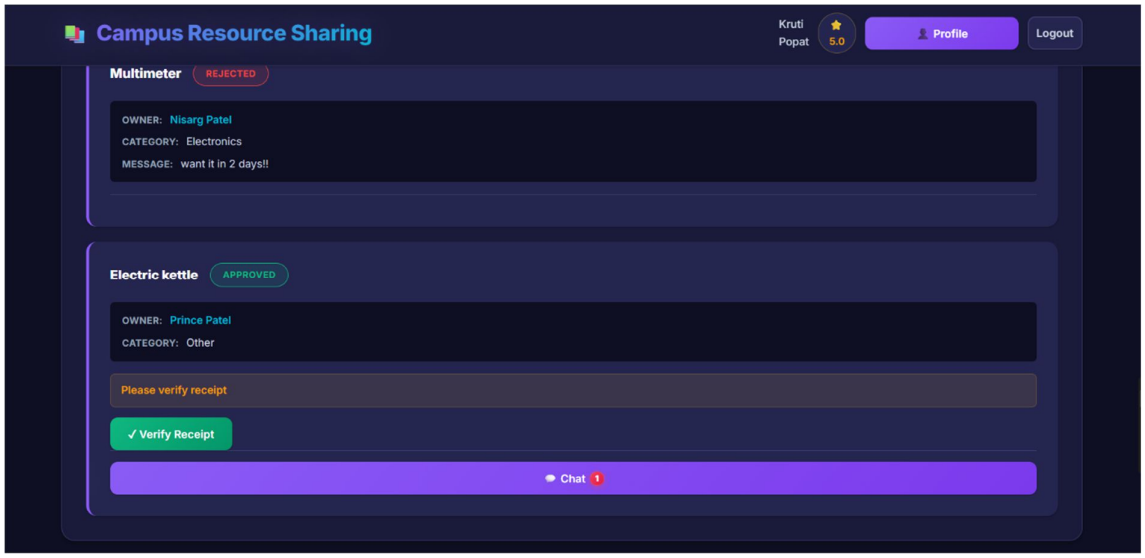

Campus Resource Sharing
Prince Patel
Profile
Logout
My Resources
My Requests
Received Requests
My Resources
Electric kettle
OTHER
For Sale: ₹800
Condition: Good
used for 1 year!
Status: Available

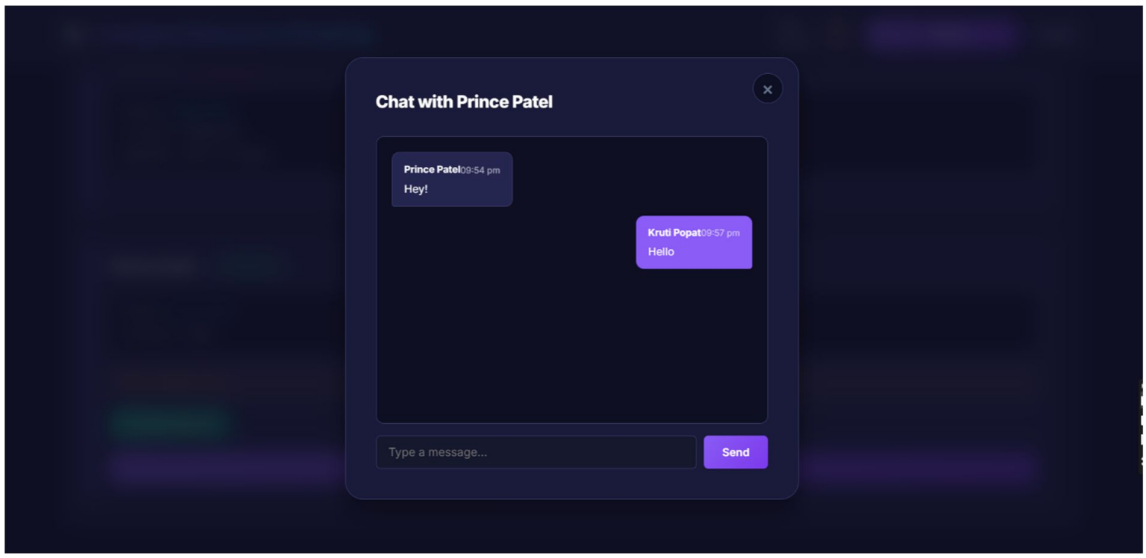

Request Resource
MESSAGE (OPTIONAL)
Add a message to the owner...
Send Request

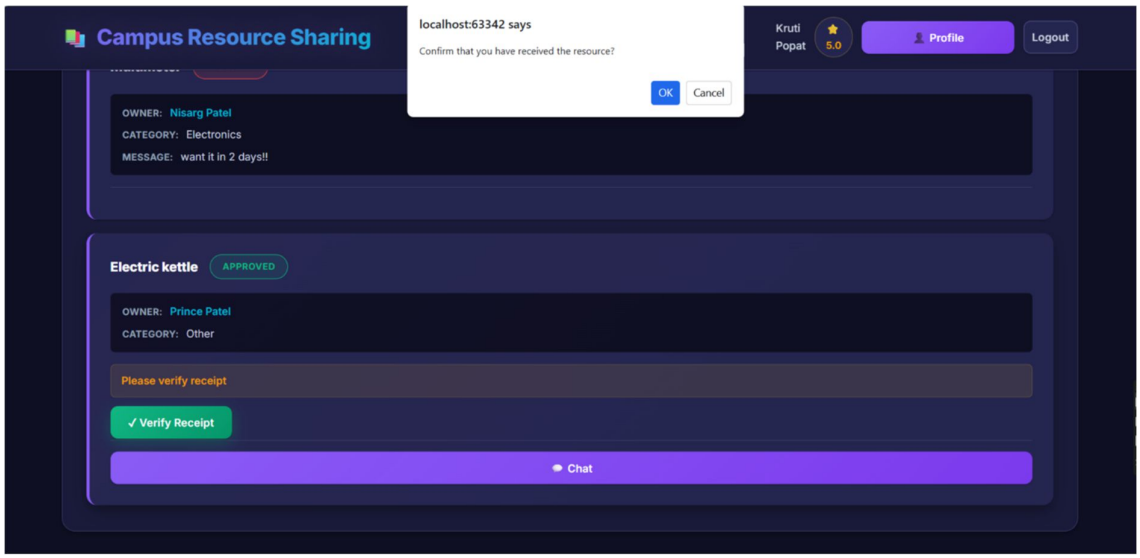

Campus Resource Sharing
Prince Patel
Profile
Logout
Owner: Kruti Popat 5.0
Already Requested
Owner: Nisarg Patel 5.0
Request
My Resources
My Requests
Received Requests 1
Received Requests
Electric kettle
PENDING
REQUESTER: Kruti Popat
CATEGORY: Other
Approve
Reject

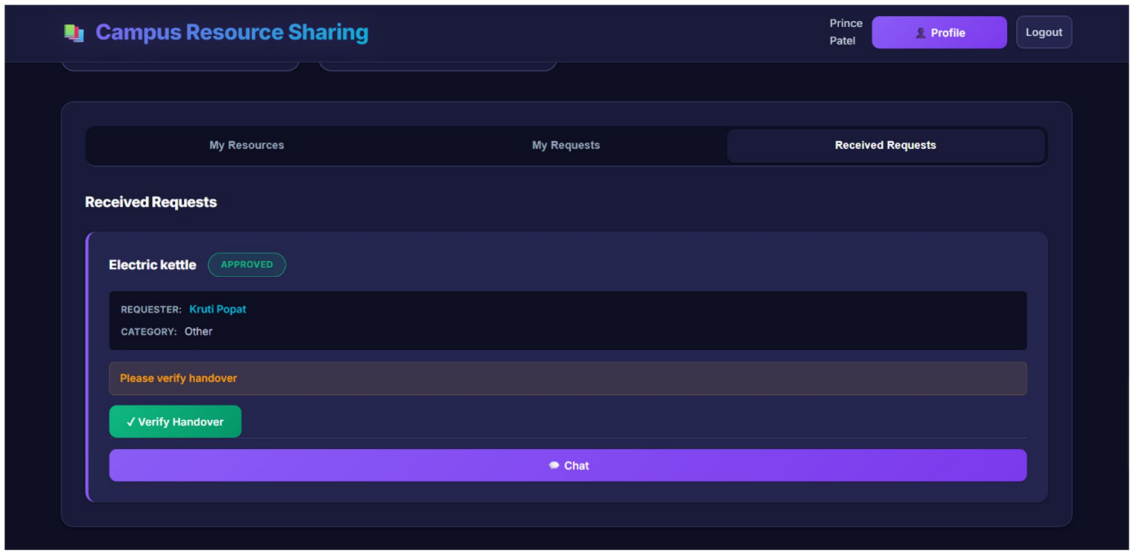

Campus Resource Sharing
Prince Patel
Profile
Logout
My Resources
My Requests
Received Requests
Received Requests
Electric kettle
APPROVED
REQUESTER: Kruti Popat
CATEGORY: Other
Please verify handover
✓ Verify Handover
Chat

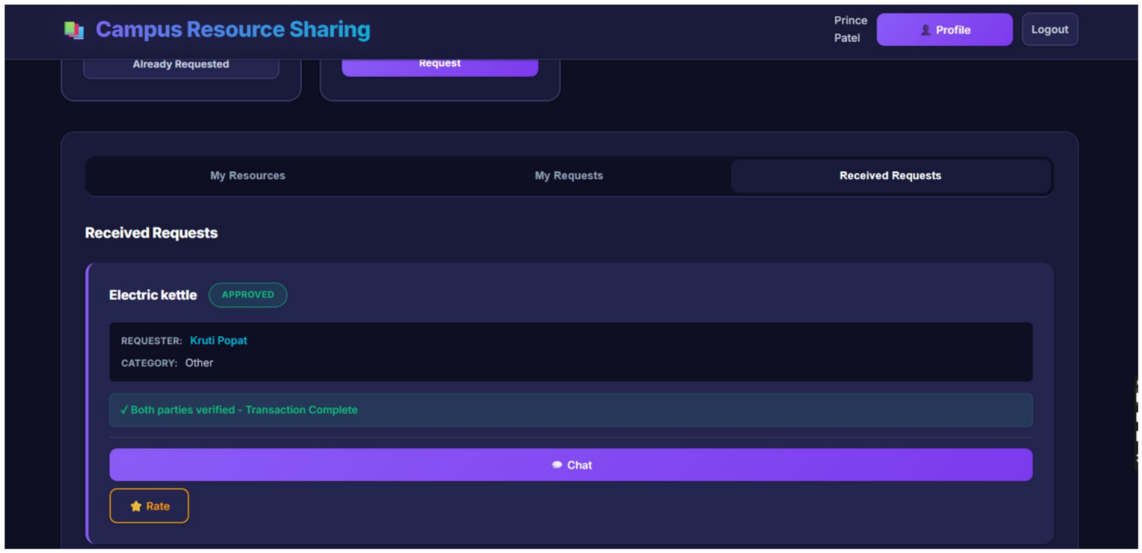

Chat with Kruti Popat
Prince Patel 09:54 pm
Hey!
Type a message...
Send

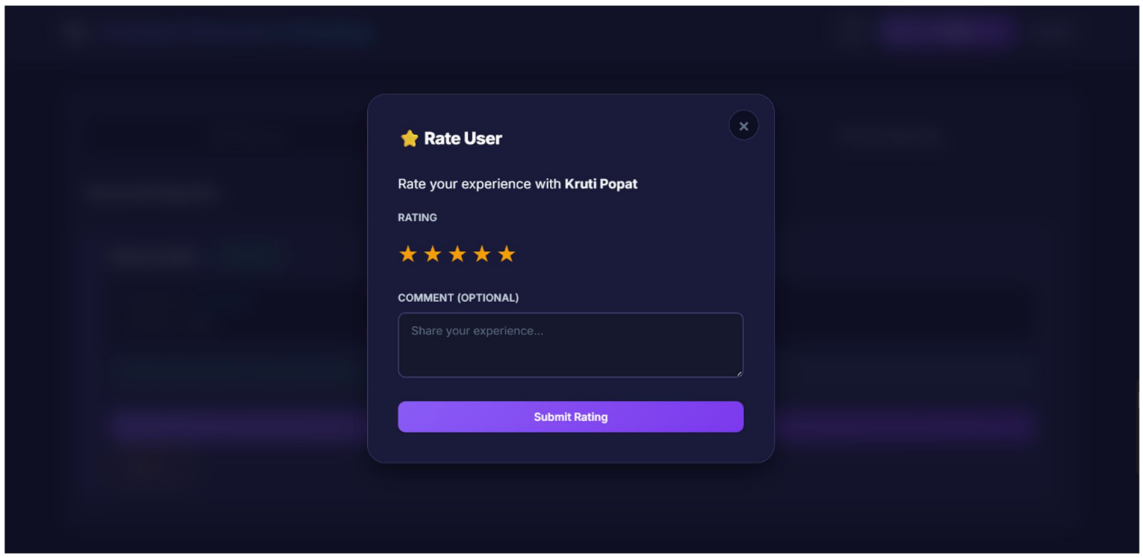

Campus Resource Sharing
Kruti Popat
5.0
Profile
Logout
Multimeter
REJECTED
OWNER: Nisarg Patel
CATEGORY: Electronics
MESSAGE: want it in 2 days!!
Electric kettle
APPROVED
OWNER: Prince Patel
CATEGORY: Other
Please verify receipt
✓ Verify Receipt
Chat 1

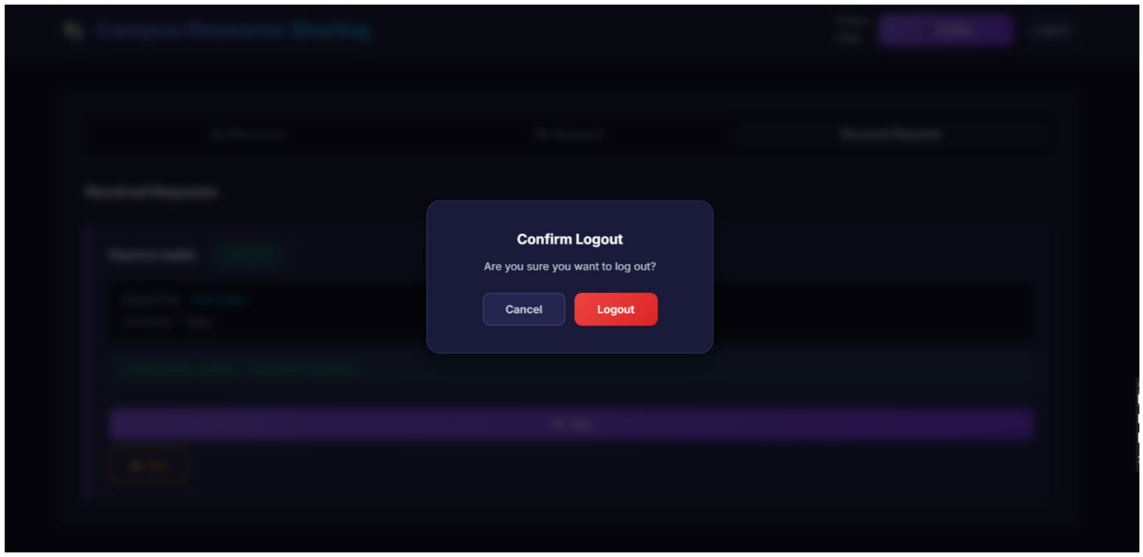

Chat with Prince Patel
Prince Patel 09:54 pm
Hey!
Kruti Popat 09:57 pm
Hello
Type a message...
Send


Campus Resource Sharing
localhost:63342 says
Confirm that you have received the resource?
OK
Cancel
Kruti Popat
5.0
Profile
Logout
OWNER: Nisarg Patel
CATEGORY: Electronics
MESSAGE: want it in 2 days!!
Electric kettle
APPROVED
OWNER: Prince Patel
CATEGORY: Other
Please verify receipt
✓ Verify Receipt
Chat


Campus Resource Sharing
Prince Patel
Profile
Logout
My Resources
My Requests
Received Requests
Received Requests
Electric kettle
APPROVED
REQUESTER: Kruti Popat
CATEGORY: Other
Please verify handover
✓ Verify Handover
Chat


Campus Resource Sharing
Prince Patel
Profile
Logout
Already Requested
Request
My Resources
My Requests
Received Requests
Received Requests
Electric kettle
APPROVED
REQUESTER: Kruti Popat
CATEGORY: Other
✓ Both parties verified - Transaction Complete
Chat
Rate


Rate User
Rate your experience with Kruti Popat
RATING
COMMENT (OPTIONAL)
Share your experience...
Submit Rating


Confirm Logout
Are you sure you want to log out?
Cancel
Logout

CHAPTER 9 LIMITATION AND FUTURE ENHANCEMENT

9.1 LIMITATIONS

Passwords stored without hashing (production would use BCrypt). No JWT or session token; frontend relies on localStorage. No image upload for resources. Chat uses polling, not WebSockets. OTP stored in memory (not suitable for multi-instance deployment). No payment integration. No push notifications. Limited search (category filter only). No admin panel for moderation.

9.2 FUTURE ENHANCEMENT

Implement BCrypt password hashing. Add JWT-based authentication. Support image upload for resources. Replace chat polling with WebSockets. Use Redis for OTP storage in distributed setup. Integrate payment gateway (Razorpay/Stripe). Add full-text search. Implement push notifications. Add admin dashboard. Add reporting (disputes, abuse). Mobile app (React Native / Flutter).

CHAPTER 10 CONCLUSION AND DISCUSSION

10.1 CONCLUSIONS AND FUTURE ENHANCEMENT

The Campus Resource Sharing Platform successfully delivers a functional system for peer-to-peer resource sharing within a campus. The system integrates user management with OTP verification, resource listing, request workflow, chat, mutual verification, and ratings. The use of Spring Boot, MySQL, and vanilla JavaScript demonstrates a complete full-stack development cycle. Future enhancements (security, real-time chat, images, payments) would strengthen the platform for production use.

10.2 DISCUSSION

10.2.1 Self Analysis of Project Viability

The project is viable for campus deployment. The technology choices are appropriate, the architecture is maintainable, and the features address real needs. Scalability can be improved with caching and connection pooling. Security hardening is recommended before production.

10.2.2 Problems Encountered and Possible Solutions

Problem: CORS blocking frontend requests. Solution: Configure CORS in CorsConfig to allow frontend origin. Problem: OTP email delivery failure. Solution: Use reliable SMTP (e.g., Brevo) and verify credentials. Problem: Chat not updating in real time. Solution: Implement polling (2s interval) or migrate to WebSockets. Problem: Duplicate requests. Solution: Validate no existing PENDING request for same resource and requester.

10.2.3 Summary of Project Work

The project encompassed requirements analysis, database design, backend development with Spring Boot, frontend development with HTML/CSS/JS, integration of OTP verification and email service, implementation of ratings, testing, and documentation. The delivered system meets the stated objectives and provides a foundation for future enhancements.

REFERENCES

- **Spring**, *Spring Boot Documentation*.
Available at: https://spring.io/projects/spring-boot

- **Oracle Corporation**, *Java Documentation*.
Available at: https://docs.oracle.com

- **Oracle Corporation**, *MySQL Documentation*.
Available at: https://dev.mysql.com/doc

- **Mozilla Foundation**, *MDN Web Docs for HTML, CSS, and JavaScript*.
Available at: https://developer.mozilla.org

- **Stack Overflow**, Programming discussions and solutions.
Available at: https://stackoverflow.com

- **GeeksforGeeks**, Tutorials on web development and databases.
Available at: https://www.geeksforgeeks.org

- **W3Schools**, Tutorials for HTML, CSS, JavaScript, and databases.
Available at: https://www.w3schools.com